Live Kalman Filtering on real data

Ilian Khiter

March/April 2026

Contents

1 Introduction **2**
1.1 Motivations . . . 2
1.2 Document's presentation . . . 2

2 Mathematical Theory and Framework **2**
2.1 Ornstein Uhlenbeck Process . . . 2
2.2 OU Process Discretisation . . . 3
2.3 State-Space Formulation of the Discretised OU Process . . . 3
2.4 Kalman Filter . . . 4

3 Implementation **5**
3.1 Interactive Brokers API . . . 5
3.2 Application Architecture . . . 5
3.3 Kalman Filter Initialization . . . 6
3.4 Real-Time Filtering Loop . . . 7
3.5 Hyperparameter Tuning . . . 8

4 Conclusion **9**

1 Introduction

1.1 Motivations

I'm currently a student in CentraleSupélec, looking for an quant internship for September 2026. This means that, while sending my resumes and trying to apply everywhere I could, I still try to have more and more knowledge in this field to prepare for the interviews and for my future career.

I wanted to implement some real signals for quant, based on real live data. I searched for relevant projects that could be accessible, and I found something called Kalman Filtering. I already saw such a method in other engineering courses, and I did not know at the time that it could be applied to finance. I started this project to learn more about this method, its implementation, and its limits.

1.2 Document's presentation

This document will present the overall theory behind the project, the implementation, and the creation of the app based on Interactive Brooker's API. The choice of this API will be presented later.

2 Mathematical Theory and Framework

2.1 Ornstein Uhlenbeck Process

Since we will be working on delayed SPY's data, (explained in further sections), we need to fit a model to this market. The literature says that this market exhibed a short-term mean reversion behaviour. Thats why we'll use the Ornstein Uhlenbeck (OU) process for our implementation.

The OU process is the unique solution of the following Stochastic Differential Equation (SDE), with dW_t a classic Wiener process:

$$dX_t = \kappa(\mu - X_t)dt + \sigma dW_t, \quad X_0 = x_0$$

where the several parameters have the following interpretation:

Param.	Name	Interpretation
$\kappa > 0$	Mean Reversion Speed	Speed of the return toward μ
$\mu \in \mathbb{R}$	Long Term Mean	Equilibrium Level
$\sigma > 0$	Volatility	Amplitude of the fluctuations around the trend

The analytic solution of the OU Process is the following:

$$X_t = \mu + (x_0 - \mu)e^{-\kappa t} + \sigma \int_0^t e^{-\kappa(t-s)} dW_s$$

Proof
By letting $u_t = e^{\kappa t} X_t$, we get by Itô's Formula:

$$du_t = \kappa e^{\kappa t} X_t dt + e^{\kappa t} X_t dt + e^{\kappa t}(\kappa(\mu - X_t)dt + \sigma dW_t)$$

Giving us:

$$du_t = \kappa \mu e^{\kappa t} dt + \sigma e^{\kappa t} dW_t$$

We can integrate this last expression between 0 and t

$$u_t = u_0 + \kappa\mu \int_0^t e^{\kappa s} ds + \sigma \int_0^t e^{\kappa s} dW_s$$

And by recalling $X_t = u_t e^{-\kappa t}$:

$$X_t = \mu + (x_0 - \mu)e^{-\kappa t} + \sigma \int_0^t e^{-\kappa(t-s)} dW_s$$

2.2 OU Process Discretisation

Since our final goal for the filter is to "predict" the next N closed price through a Kalman Filter (explained later), we want to compute the law of $X_{t+\Delta t}|X_t$, which is given by:

$$X_{t+\Delta t}|X_t \sim \mathcal{N}\left(\mu + (X_t - \mu)e^{-\kappa\Delta t}, \frac{\sigma^2}{2\kappa}[1 - e^{-2\kappa\Delta t}]\right)$$

Proof

Using our last result applied from time t, we'll have:

$$X_{t+\Delta t} = \mu + (X_t - \mu)e^{-\kappa\Delta t} + \sigma \int_t^{t+\Delta t} e^{-\kappa(t+\Delta t-u)} dW_u$$

The first part of the expression is $\mathcal{F}_t$ *mesurable. The second one is independant of* $\mathcal{F}_t$*, and is a centered gaussian random variable. Its variance is, giving by Itô's isometry:*

$$Var(\sigma \int_t^{t+\Delta t} e^{-\kappa(t+\Delta t-u)} dW_u) = \sigma^2 \int_t^{t+\Delta t} e^{-2\kappa(t+\Delta t-u)} du = \frac{\sigma^2}{2\kappa}[1 - e^{-2\kappa\Delta t}]$$

and we have our result.

Now that we have our discretisation, can look forward to the Kalman Filter and the vector state-space.

2.3 State-Space Formulation of the Discretised OU Process

The state-space formulation is the key step that allows us to apply the Kalman Filter to our problem. The idea is to rewrite the discretised OU process as a system of two equations — one describing how the hidden state evolves, and one linking the hidden state to what we observe.

The parameters (κ, μ, σ) are unknown and evolve slowly over time. We gather them in a state vector:

$$x = \begin{pmatrix} \kappa \\ \mu \\ \sigma \end{pmatrix}$$

We model the parameters as a random walk — they drift slowly between two consecutive bars. Hence, the above state vector will be adapted thanks to the following state equation:

$$x_{t+1} = x_t + w_t, \quad \text{with} \quad w_t \sim \mathcal{N}(0, Q)$$

where Q is the process noise covariance matrix, controlling how fast the parameters are allowed to drift. Its value will be detailed further.

At each bar closing, we observe the price S_{t+1}, which is linked to the state via the discretised OU dynamics, which led to the following observation equation:

$$S_{t+1} = h(x_t, S_t) + v_t, \quad \text{with} \quad v_t \sim \mathcal{N}(0, R)$$

where h is the observation function:

$$h(x_t, S_t) = e^{-\kappa\Delta t} S_t + \mu(1 - e^{-\kappa\Delta t})$$

and R is the observation noise variance, representing market microstructure noise.

2.4 Kalman Filter

The Kalman Filter is a recursive algorithm that estimates the hidden state x_t from noisy observations, by optimally combining the prediction from the state equation and the information brought by each new observation.

It operates in two steps at each time t:
Predict Step: Before observing S_{t+1}, we propagate the state estimate and its uncertainity:

$$\hat{x}_{t|t-1} = \hat{x}_{t-1}$$

$$P_{t|t-1} = P_{t-1} + Q$$

where P_t is the state covariance matrix, representing our uncertainty on $\hat{x}_t$.

Update Step: Once S_{t+1} is observed, we correct the prediction using the innovation y_t (the gap between what we observed and what we predicted):

$$y_t = S_{t+1} - h(\hat{x}_{t|t-1}, S_t)$$

The Kalman gain K_t weights the correction optimally:

$$K_t = P_{t|t-1} H_t^T (H_t P_{t|t-1} H_t^T + R)^{-1}$$

And we update the state and covariance:

$$\hat{x}_t = \hat{x}_{t|t-1} + K_t y_t$$

$$P_t = (I - K_t H_t) P_{t|t-1}$$

where $H_t = \frac{\partial h}{\partial x}|_{\hat{x}_{t|t-1}}$ is the Jacobian of the observation function. Here, the Jacobian is given by:

$$H_t = [\Delta t . e^{-\kappa\Delta t}(\mu - S_t) \quad 1 - e^{-\kappa\Delta t} \quad 0]$$

Proof
The jacobian of h with respect to $x_t = [\kappa, \mu, \sigma]^T$ *is :*

$$H_t = \begin{bmatrix} \frac{\partial h}{\partial \kappa} & \frac{\partial h}{\partial \mu} & \frac{\partial h}{\partial \sigma} \end{bmatrix}$$

The several components of the matrix are given by:

$$\frac{\partial h}{\partial \kappa} = -\Delta t e^{-\kappa\Delta t} S_t + \mu \Delta t e^{-\kappa\Delta t}$$

$$\frac{\partial h}{\partial \mu} = 1 - e^{-\kappa\Delta t}$$

$$\frac{\partial h}{\partial \sigma} = 0$$

hence giving us the result.

3 Implementation

3.1 Interactive Brokers API

The data feed for this project is provided by the Interactive Brokers (IBKR) API, through their Python client library ibapi. Interactive Brokers is one of the leading brokerage firms in the world, known for its robust and well-documented API that gives developers direct access to market data, order management, and account information. The API follows a client-server architecture — the user runs the Trader Workstation (TWS) software locally, which acts as a gateway between the Python client and IBKR's servers. All API requests and responses are routed through TWS via a socket connection.

The Python client is structured around two base classes that must be inherited simultaneously: EClient, which handles outgoing requests to TWS, and EWrapper, which handles incoming callbacks from TWS. This means that every API call is asynchronous — when you request market data, the response does not come back as a return value, but instead triggers a callback method that you override in your implementation. For example, requesting market data via reqMktData will eventually trigger the tickPrice callback each time a new price tick arrives. This event-driven architecture requires careful threading management, as the API runs its own event loop in a dedicated thread.

The underlying asset chosen for this project is SPY — the SPDR S&P 500 ETF Trust — which tracks the S&P 500 index. SPY is one of the most liquid instruments in the world, with extremely tight bid-ask spreads and consistently high trading volume throughout the session. These properties make it an ideal candidate for mean-reversion strategies, as the presence of market makers and arbitrageurs tends to keep the price close to its fair value at short timescales.

Since this project was developed without a funded brokerage account, we use delayed market data (15-minute delay), which IBKR provides for free to all users. While this delay prevents any live trading application, it is entirely sufficient for our purposes — the goal here is to validate the Kalman filtering methodology on realistic market data, not to execute trades. The connection is established on port 7497, which corresponds to IBKR's paper trading TWS instance, and the market data type is explicitly set to delayed via reqMarketDataType(3).

Upon connection, two data streams are requested simultaneously. First, live market data is requested via reqMktData, which streams real-time bid (tick type 66), ask (tick type 67), and last price (tick type 68) through the tickPrice callback. From the bid and ask prices, we compute the mid-price at each tick as $\frac{P_{bid}+P_{ask}}{2}$, which serves as our primary price signal. Second, historical data is requested via reqHistoricalData, fetching the last hour of 30-second MIDPOINT bars through the historicalData callback. These historical bars are essential for the initialization of the Kalman filter, as they provide the empirical estimates of μ_0 and σ_0 before any live data arrives.

3.2 Application Architecture

The application is structured around two main classes that operate concurrently via Python's threading module.

The first class, IBApp, inherits from both EClient and EWrapper and handles all communication with TWS. It stores the incoming market data — bid, ask, last price, mid-price history, and historical bars — as instance attributes that are updated at each callback. A set of threading.Event objects is used to synchronize the initialization sequence: hist_done is set when the

historical data has been fully received, signaling to the rest of the application that the Kalman filter can be initialized.

The second class, LiveInterfaceKalman, handles the user interface and the real-time processing logic. It is built on top of Tkinter, Python's standard GUI library, and uses Matplotlib embedded in the Tkinter window via FigureCanvasTkAgg to render the candlestick chart. The interface is organized into four frames — a control panel, a market data panel, a chart panel, and a log panel.

The threading architecture is the following. Upon clicking the connect button, three concurrent threads are launched:

- The IB thread runs IBApp.run(), the API event loop that listens for incoming callbacks from TWS
- The Kalman thread runs initKalman(), which waits for hist_done to be set, then instantiates the KalmanEstimator and runs the filter on the historical bars to warm-start it
- The Tkinter main loop runs on the main thread and drives the UI refresh cycle via root.after() callbacks at 30ms intervals

This separation ensures that the API event loop and the filter initialization never block the UI, keeping the interface responsive at all times. Bar construction. The mid-price ticks streamed by the API are aggregated into 30-second OHLC bars in the update_OHLC method, which is called every 30ms by the Tkinter scheduler. Each tick is assigned to a bar based on its timestamp, floored to the nearest 30-second boundary. When a new bar starts, the previous one is closed and appended to the bar deque. If gaps are detected between consecutive bars — for instance due to periods of inactivity — synthetic bars are inserted at the last close price to maintain a continuous time series.

3.3 Kalman Filter Initialization

A critical step in any Kalman filter implementation is the initialization of the state vector $\hat{x}_0$ xand the covariance matrices P_0, Q and R. Poor initialization can lead to slow convergence or numerical instability, so we use the historical bars to derive empirical estimates before any live data arrives.

State vector initialization. The initial state vector is set as:

$$\hat{x}_0 = \begin{pmatrix} \kappa_0 \\ \mu_0 \\ \sigma_0 \end{pmatrix}$$

The initial long term mean μ_0 is is estimated as the sample mean of the historical close prices:

$$\mu_0 = \frac{1}{N} \sum_{i=1}^{N} S_i$$

The initial volatility σ_0 is estimated from the log-returns of the historical close prices:

$$\sigma_0 = \text{std}(\log(S_{i+1}) - \log(S_i))$$

The initial mean-reversion speed κ_0 cannot be easily estimated from a short window of data, so it is set to a fixed value of $\kappa_0 = 0.5$ as a starting point. The filter will then update at each closing bar.

For the several matrices, we begin by setting parameters arbitrarily. We'll see in the "hyper-parameter tuning" section another way to set up those matrices. For now, after some manual calibration, I found that the following values were a good fit for our model:

$$P_0 = \text{diag}(10^{-2}, 1, 10^{-2}), \quad Q = \text{diag}(0.3, 1, 10^{-2})$$

The observation noise variance R is estimated empirically from the historical data as the variance of the first differences of close prices, scaled by a factor of 10 to account for the fact that the mid-price is already a smoothed estimate of the true price:

$$R = 10 \times \text{var}(S_{i+1} - S_i)$$

Once the Kalman Estimator is instantiated with these initial values, the filter is immediately run on the full historical bar sequence before any live data arrives. This warm start ensures that by the time the first live bar closes, the filter has already converged to a reasonable estimate of the parameters, rather than starting cold from $\hat{x}_0$

3.4 Real-Time Filtering Loop

The integration of the Kalman filter into the live application required several engineering choices to ensure correct synchronization between the data feed, the filter, and the UI.

Bar closing detection. The update_OHLC method is called every 30ms by the Tkinter scheduler. It processes all mid-price ticks accumulated since the last call, and detects a bar closing when a new tick arrives with a timestamp belonging to a different 30-second window than the current bar. At that moment, the closed bar is appended to the bar deque, and the Kalman filter is immediately triggered with S_{prev} and S_{new} being the close prices of the previous and current bars respectively. If gaps are detected in the time series — for instance during periods of low activity — synthetic bars are inserted at the last close price to maintain continuity.

Threading and UI responsiveness. A key constraint of the Tkinter framework is that all UI updates must happen on the main thread. The Kalman initialization, however, needs to wait for the historical data to arrive before it can run — which could take several seconds. To avoid blocking the UI during this wait, the initialization is run in a dedicated daemon thread that waits on the hist_done event. Once the historical data is ready, the filter is instantiated and the warm start is executed entirely in this background thread, leaving the main thread free to keep the UI responsive.

Warm start alignment. After the warm start, the kalmanDeque contains one μ estimate per historical bar. On the chart, these points must be aligned with their corresponding bars. Since both the historical bars and the deque are ordered chronologically, the last point in the deque is always aligned with the last historical bar, and the alignment is maintained by slicing both sequences to the same length.

The following image shows the live chart while running the app. We have as described earlier, the historical OHLC candles bar and the live updated one, the historical mean μ which is represented by the blue line, and the last estimated $\hat{\mu}$ represented with the grey line, as well as orange and violet dots. Those dots represents the behaviour of our Kalman implementation. The orange ones are the historical estimators of the prices, and the violet ones are the 5-bar ahead forecast on the price trajectory. Those dots show well the behavior of the model to

be elastic to the estimated mean, as we see with the curve made by the forecast on the price trajectory.

3.5 Hyperparameter Tuning

The Kalman filter depends on three covariance matrices that must be specified by the user: the initial state covariance P_0, the process noise covariance Q, and the observation noise variance R. While P_0 and R can be initialized from the historical data in a principled way, the tuning of Q is less straightforward and has a critical impact on the filter's behavior.

The ratio Q/R is the central tuning parameter of the filter. Intuitively, a large Q relative to R means the filter believes the parameters drift quickly and should closely follow new observations — making it reactive but noisy. A small Q relative to R means the filter trusts its prior estimates more than the new data — making it stable but slow to adapt. In practice, this tradeoff manifests visually on the chart: a large $Q[1,1]$ makes orange dots track the price closely, while a small $Q[1,1]$ keeps them close to the historical mean. While this approach is pragmatic and common in engineering applications, it lacks theoretical guarantees and may not generalize well to different market regimes. A more rigorous approach would be to optimize Q and R by maximizing the marginal log-likelihood of the observations. At each step, the innovation y_t follows a Gaussian distribution with zero mean and variance $S_t = H_t P_{t|t-1} H_t^T + R$. The log-likelihood is then:

$$\mathcal{L} = -\frac{1}{2} \sum_t (\log(|S_t|) + y_t^T S_t^{-1} y_t)$$

Maximizing this expression over Q and R using scipy.optimize would yield data-driven hyperparameters that are optimal in the maximum likelihood sense. Alternatively, the Expectation-Maximization (EM) algorithm provides an iterative procedure that alternates between running the filter and smoother to estimate the hidden states, and analytically updating Q and R in closed form. Both approaches are left as perspectives for future work.

In this implementation, Q was tuned manually by iterating on the live chart and adjusting the diagonal entries until the filter's behavior matched the desired balance between reactivity and stability.

4 Conclusion

This project presented the full pipeline of a live Kalman filtering application applied to real market data, from the mathematical foundations to the practical implementation. Starting from the Ornstein-Uhlenbeck process as a model for the short-term mean-reverting dynamics of SPY, we derived the discretised state-space formulation that allowed us to cast the parameter estimation problem as an Extended Kalman Filter. The filter was then integrated into a live trading interface connected to Interactive Brokers' API, providing real-time estimates of the OU parameters (κ, μ, σ) and a 5-bar ahead forecast of the price trajectory.

This work should be understood as a first prototype rather than a finished product. The results are encouraging — the filter converges quickly after the warm start, the μ estimates track the price dynamics in a visually meaningful way, and the forecast provides an interpretable mean-reversion signal. However, several limitations remain.

The hyperparameter tuning is currently done by hand, which is neither rigorous nor robust across different market regimes. The observation model could be enriched — for instance by incorporating volume or bid-ask spread as additional signals. The EKF itself could be replaced by an Unscented Kalman Filter, which handles non-linearities more gracefully without requiring an explicit Jacobian. Finally, the current implementation does not include any backtesting framework, which would be necessary to assess the actual predictive power of the filter.

These improvements are not merely theoretical perspectives — they are concrete next steps that will be implemented in future iterations of this project. The MLE-based hyperparameter optimization and the UKF are particularly high on the roadmap, as they would significantly strengthen both the theoretical rigor and the practical robustness of the approach.

In a broader sense, this project was a valuable exercise in bridging stochastic calculus, signal processing, and software engineering in a real-time financial context — a combination that lies at the heart of modern quantitative finance.